

Jadavpur University



Computer Networks

Lab Report

NAME : ANKIT SHAW

ROLL NO. : 002310501032

YEAR : BCSE UG – III (A1)

SESSION : 2023 – 27

ASSIGNMENT : 0 4

Objective

The objective of this assignment is to implement and simulate the **Code Division Multiple Access (CDMA)** protocol, a fundamental channel access method that allows multiple stations to transmit data simultaneously over a shared physical medium. This will be accomplished by developing a multi-threaded simulation where each sender is assigned a unique, orthogonal code derived from a Walsh-Hadamard matrix.

The core task is to demonstrate how each sender can encode its data bit (1, 0, or silence) by spreading it with its assigned Walsh code, contribute the resulting signal to a common channel, and how a corresponding receiver can then perfectly reconstruct the original data bit from the composite signal. The simulation architecture will consist of multiple sender threads, multiple receiver threads, and a shared channel object that aggregates the signals from all active senders.

The final goal is to verify that the mathematical principle of orthogonality allows each receiver to isolate and decode its intended signal, even amidst the "noise" created by other simultaneous transmissions, thereby proving the efficacy of the CDMA scheme.

Implementation

Code Division Multiple Access (CDMA) is a channel access method where multiple transmitters can send information simultaneously over a single communication channel. This is achieved by using spread-spectrum technology and a special coding scheme where each transmitter is assigned a unique code. An analogy is a room full of people where each pair is speaking a different language; even though everyone is talking at once, you can still understand the person speaking your language.

In this implementation, the unique codes are generated from a Walsh-Hadamard matrix. These codes have a crucial mathematical property: they are perfectly orthogonal. This means that when you take the inner product (or dot product) of any two distinct codes from the set, the result is zero. It is this property that allows a receiver to "tune in" to its specific sender's signal and filter out the combined noise from all other simultaneous transmissions.

The simulation was implemented in Python, leveraging the threading library to model the concurrent transmission and reception of signals from multiple network stations. The code is organized into four logical files: `main.py` (the simulation orchestrator), `sender.py` (logic for encoding and transmitting a signal), `receiver.py` (logic for decoding a signal from the channel), and `channel.py` (which manages the shared channel state).

main.py

```
import threading import
random
from channel import channel_signal, NUM_STATIONS from
sender import sender
from receiver import receiver # renamed to avoid import error
def generate_walsh_table(n):
    """Generates a Walsh-Hadamard matrix of size n x n (n must be a power of
2)."""
    if n ==
1:
        return [[1]]

    W_half = generate_walsh_table(n // 2)
    W_full = []
    for i in
range(n//2):
        W_full.append(W_half[i] + W_half[i])
    for i in
range(n // 2):
        neg_W_half_row = [-x for x in W_half[i]]
        W_full.append(W_half[i] + neg_W_half_row)
    return
W_full
def next_power_of_2(x):
    """Returns the next power of 2 greater than or equal to x."""
    power = 1
    while
power < x:
        power
*= 2
    return power
def main():
    """Runs the CDMA simulation."""

    actual_stations = NUM_STATIONS
    N = next_power_of_2(actual_stations) # next power of 2
    print(f'Actual Stations: {actual_stations}, Using Walsh size: {N}')
    # Initialize channel
    channel_signal.extend([0] * N)

    # Generate Walsh codes
    walsh_table = generate_walsh_table(N)

    # Assign random 0/1/None (None = silence) for actual stations
    data_to_send =
[random.choice([0, 1, None]) for _ in range(actual_stations)]

    # Pad extra stations with None (silence)
    for _ in
range(N - actual_stations):
        data_to_send.append(None)

    decoded_data = [None] * N
    # Display setup
    print("--- CDMA Simulation Setup ---")
    for i in range(N):
        print(f'Station {i}:
Walsh Code = {walsh_table[i]}, Data to Send =
```

```

{data_to_send[i]})    print("-" * 30
+ "\n")

# Transmission phase
print("--- Transmission Phase ---")    sender_threads = []    for i in range(N):
thread = threading.Thread(target=sender, args=(i, walsh_table[i], data_to_send[i]))
    sender_threads.append(thread)    thread.start()
    for thread in sender_threads:
thread.join()

print(f"\nFinal Combined Signal on Channel: {channel_signal}\n")
# Reception phase
print("--- Reception Phase ---")    receiver_threads = []    for i in range(N):    thread
= threading.Thread(target=receiver, args=(i, walsh_table[i], decoded_data))
    receiver_threads.append(thread)    thread.start()
    for thread in receiver_threads:
thread.join()

# Verification    print("\n" + "-" *
30)    print("--- Verification ---")

    correct_decodes = 0    for i in range(N):    sent = data_to_send[i]    decoded
= decoded_data[i]    if sent == decoded or (sent is None and decoded == 'silence'):
        print(f'Station {i}: SUCCESS! Sent: {sent}, Decoded: {decoded}')
correct_decodes += 1    else:    print(f'Station {i}: FAILURE! Sent: {sent}, Decoded:
{decoded}')

    print("-" * 30)    if correct_decodes == N:    print("All data was transmitted and
received successfully!")    else:    print(f'Simulation failed. {N - correct_decodes} error(s)
occurred.') if __name__ == "__main__":    main()

```

receiver.py

```

from channel import channel_signal
def receiver(receiver_id, walsh_code, decoded_results):
    """Decodes data from the channel using the receiver's Walsh code."""
    # Calculate inner product (correlation)
    inner_product = sum(c * s for c, s in zip(walsh_code, channel_signal))
    # Normalize to get bipolar value
    decoded_value = inner_product / len(walsh_code)

    # Convert back to bit or silence    if decoded_value > 0:
original_bit = 1    elif decoded_value < 0:    original_bit =
0    else:    original_bit = 'silence' # No transmission

    print(f'Receiver {receiver_id}: Decoded '{original_bit}'")    decoded_results[receiver_id]
= original_bit

```

sender.py

```
from channel import channel_lock, channel_signal
def sender(sender_id, walsh_code, data_bit):
    """Encodes a data bit and adds it to the channel.""" # Convert
    data bit to bipolar (+1, -1) or 0 for silence    if data_bit == 1:
data_bipolar = 1    elif data_bit == 0:        data_bipolar = -1
    else: # None or 's' indicates silence        data_bipolar = 0
    # Create the sender's signal by multiplying data with its Walsh code    encoded_signal
    = [chip * data_bipolar for chip in walsh_code]

    print(f"Sender {sender_id}: Sending '{data_bit}' as {encoded_signal}")
    # Add the signal to the shared channel safely    with
channel_lock:    for i in range(len(channel_signal)):
        channel_signal[i] += encoded_signal[i]
```

channel.py

```
import threading
# The number of stations/senders
NUM_STATIONS = 6

# Shared channel signal and lock for thread safety. channel_signal = []
channel_lock = threading.Lock()
```

Results :

The simulation successfully demonstrates the principles of Code Division Multiple Access. The program was executed with six active stations, which required the generation of an 8x8 Walsh-Hadamard matrix to assign a unique and orthogonal code to each potential sender. Each of the six active stations was randomly assigned a data bit of 1, 0, or None to represent silence.

During the transmission phase, each sender encoded its data bit into a bipolar signal (+1, -1, or 0) and multiplied it by its assigned Walsh code. All these individual signals were then summed together on a shared channel, creating a single composite signal that represented all simultaneous transmissions.

In the reception phase, each receiver successfully decoded its intended data from this composite signal. A final verification step compared the original data sent by each station with the data decoded by its corresponding receiver, confirming a 100% success rate with zero errors. This outcome validates that multiple transmissions occurred simultaneously over the shared channel without interfering with one another.

Analysis of Results :

The simulation results confirm that CDMA can successfully manage simultaneous transmissions. This success is entirely dependent on the mathematical property of orthogonality inherent in the Walsh codes. Orthogonality ensures that the inner product (or dot product) of any two different Walsh codes from the set is exactly zero.

When a receiver wants to decode a message, it performs this inner product calculation between the total composite signal on the channel and its own unique Walsh code. Because of orthogonality, the signal components contributed by all other senders (using different Walsh codes) mathematically cancel out to zero during this process. The only component that does not cancel out is the signal from the sender that used the exact same Walsh code.

This calculation results in a value which, when normalized by dividing it by the length of the code, yields either +1, -1, or 0. The receiver can then unambiguously interpret these results as the original data bit 1, 0, or silence, respectively.

Therefore, the perfect data reconstruction shown in the results is a direct consequence of this mathematical "filtering." Each receiver can effectively isolate its intended signal from the noise of all other simultaneous transmissions, proving the protocol's effectiveness. The 100% success rate in the verification step serves as a practical validation of this core principle.

Comments :

This lab provided valuable hands-on experience with **Code Division Multiple Access**, a foundational multiple access protocol. By implementing the CDMA algorithm from scratch using a multi-threaded model, I gained a deeper, practical understanding of core concepts like orthogonal codes, spread spectrum, signal encoding via Walsh-Hadamard matrices, and signal decoding using the inner product.

The assignment struck a good balance in difficulty. The primary conceptual challenge was understanding how the mathematical property of orthogonality could be practically applied to allow a receiver to perfectly isolate a signal from a composite wave containing multiple other signals. From an implementation perspective, correctly writing the recursive function to generate the Walsh-Hadamard table was a key learning outcome. Using threads to simulate the concurrent actions of senders and receivers reinforced the client-server design pattern used in previous assignments.

The problem statement was clear, and the objective to demonstrate perfect data reconstruction provided a tangible and satisfying goal.

For future improvement, the simulation could be enhanced to model a more realistic channel by introducing random noise to the composite signal. This would allow for an analysis of how robust CDMA is to interference and at what noise level the decoding process begins to fail. Additionally, comparing the properties of CDMA against other multiple access schemes like TDMA or FDMA in a simulation would provide a more comprehensive understanding of the trade-offs in channel access strategies.

